Milestone 2: UI Design

Project: Sigil
Team: The Primary Keys
Course: CS-UY 3083 Introduction to Databases, Prof. Arfaoui, NYU Tandon
Date: October 19, 2026

Team members:

- Mohammad Alyousef (mba6777)
- Khalid Alhabib (ka3456)
- Harsh Agarwal (ha2957)
- Pratyush Patel (pp2916)

Figma file with all screens: https://www.figma.com/design/lufTAQB14IdXxqn21MJkO9/Sigil---Milestone-2-UI-Design

Who wrote what: Khalid Alhabib: Student screens 2–5; Pratyush Patel: Sign-in and Instructor screens 1, 6–8; Mohammad Alyousef: Registrar screens 9–12; Harsh Agarwal: Platform choice, target users, design rationale, Verifier screens 13–15, Mobile views 16.

1. Platform Choice

Sigil will be built as a web application: a responsive single-page app in the browser, talking to a REST API in front of a PostgreSQL database. We compared it with a desktop app and a mobile app and chose the web for the reasons below.

Verifiers must not need an account or an install. Milestone 1 (Assumption 3) says employers verify a transcript without registering. A web link or QR code that opens a public page is the only option that needs no install and no login.

Four very different user groups. Students, instructors, registrar staff and outside verifiers all use the same system on whatever device they have. A browser runs on lab PCs, laptops and phones from one codebase.

Verification can run on the client. The browser's Web Crypto API can recompute SHA-256 leaf hashes, walk the Merkle path and check the university's signature locally. Our server only delivers the proof data; the verdict is computed on the verifier's own machine against NYU's published key, and a downloaded proof file can be checked with no network request at all.

Fits the database work in this course. PostgreSQL lets us enforce the append-only rule with permissions: REVOKE UPDATE, DELETE ON grade. CHECK constraints cover the letter grades, and foreign keys cover supersedes_grade_id.

Alternatives we considered: Desktop app (Electron/JavaFX): Every verifier would have to install it. Rejected. Native mobile app: Grade entry, ledgers and checkpoint publishing need large tables and screens. The web version stays usable on phones for the student and verifier pages.

UI Development Environment

Layer	Tool / technology
Design tool	Figma (all 16 screens are frames in one shared file)
Front end	React 18 + TypeScript, built with Vite; React Router for the four role areas
UI layer	Our own component kit (sidebar shell, cards, tables, badges) in plain CSS variables
Crypto in browser	Web Crypto API (SHA-256 and Ed25519 signature check) for the public verifier
Back end	Node.js + Express REST API with role-based access (student / instructor / registrar)
Database	PostgreSQL 16 (append-only grade table, CHECK constraints, roles and GRANTs)
Auth	NYU NetID login (password or SSO); verifiers use the public pages with no login

2. Target Users

Sigil has four groups of users. The first three sign in with their NYU NetID. Verifiers use public pages and never need an account.

User	What they need to do	Screens
Student	Sees their full grade history, proves grades to employers, and shares only the grades they choose.	Transcript, Grade history, Create proof, My proofs, Mobile views
Instructor	Enters final grades for their courses before the deadline and requests corrections with a stated reason.	My courses, Grade entry, Request correction
Registrar staff	Approves or rejects corrections, seals each term into a signed Merkle checkpoint, and audits the ledger.	Dashboard, Correction approvals, Publish checkpoint, Audit ledger
Verifier (employer, admissions)	Checks that shared grades are genuine and current without phoning or emailing NYU and without creating an account.	Verify a proof, Verification result, Checkpoint registry, Mobile views

3. Design Overview

One shell for every role. Every signed-in screen has the same layout: a left sidebar for navigation, a top bar with the page title, breadcrumb, global search, notifications and the current term, and a content area. Users who hold two roles (a TA who is also a student, for example) do not have to learn a second layout.

Role colours match the ERD. The dark theme reuses the Milestone 1 ERD palette: teal = Student, orange = Instructor, purple = University/Registrar, green = MerkleCheckpoint, indigo = Verifier, gold = DisclosureProof. Grade is red in the ERD but rose in the UI, because red is reserved for errors and destructive actions. Each area's primary button always uses that role's colour, so the main action looks the same on every screen of an area.

Accessible contrast. Every text colour meets WCAG AA (at least 4.5:1) on every background it sits on. The dimmest text token (#8a8a9c) measures between 4.6:1 and 5.8:1, depending on the surface. Touch targets on the mobile pages are at least 44 px tall.

One consistent scenario. All screens show the same moment, October 19, 2026, with the same data. Priya Shah's Fall 2025 Databases grade was corrected and sealed in January (screens 3, 12, 14, 15). Today Julia Rivera drafts a new correction (screen 8). The registrar reviews a different pending one (screen 10) and publishes Spring 2026 v2 with two corrections approved earlier (screen 11). IDs, leaf indexes and root hashes match across screens.

Checked against Nielsen's heuristics. Visibility of system status: progress bars, autosave badge, pipeline steps, verdict banner. Match with the real world: students see 'Grade fingerprint' and 'Tree root', not column names. Error prevention: disabled Submit until all grades are valid, rule checks before approval, confirmation boxes. Recognition over recall: the current grade is shown next to every correction, and allowed grades are listed beside the roster. Help users recover from errors: every failed verification says why and what to do next (screen 16).

Status is never colour-only. Every state (Sealed, Corrected, In progress, Active, Revoked, Expired, Invalid grade) is a text badge, so the meaning survives colour blindness and grayscale printing.

Monospace for anything cryptographic or an ID. Hashes, grade_id, NetIDs, course codes and proof IDs use a monospace font, so users can compare them character by character.

Irreversible actions need confirmation. Submitting grades, publishing a checkpoint and revoking a proof cannot be undone. Each one sits behind a confirmation checkbox or a red outlined button, and the screen states the consequence first. A button that cannot be used yet is drawn greyed out, and nearby text says what will unlock it.

Sample data from the NYU Bulletin. Course codes and titles (CS-UY 3083 Introduction to Databases, CS-UY 4783 Applied Cryptography, and so on) come from the NYU Tandon bulletin, so the screens reflect real course and term structure. Student and staff names are fictional.

Numbered annotations. Each screen carries yellow numbered markers. Each number matches a row in that screen's component table below.

Screen List

No.	Screen	User	Written by
1	Sign in	All signed-in users	Pratyush Patel
2	Student: Transcript	Student	Khalid Alhabib
3	Student: Grade history	Student	Khalid Alhabib
4	Student: Create disclosure proof	Student	Khalid Alhabib
5	Student: My proofs	Student	Khalid Alhabib
6	Instructor: My courses	Instructor	Pratyush Patel
7	Instructor: Grade entry	Instructor	Pratyush Patel
8	Instructor: Request a grade correction	Instructor	Pratyush Patel
9	Registrar: Dashboard	Registrar staff	Mohammad Alyousef

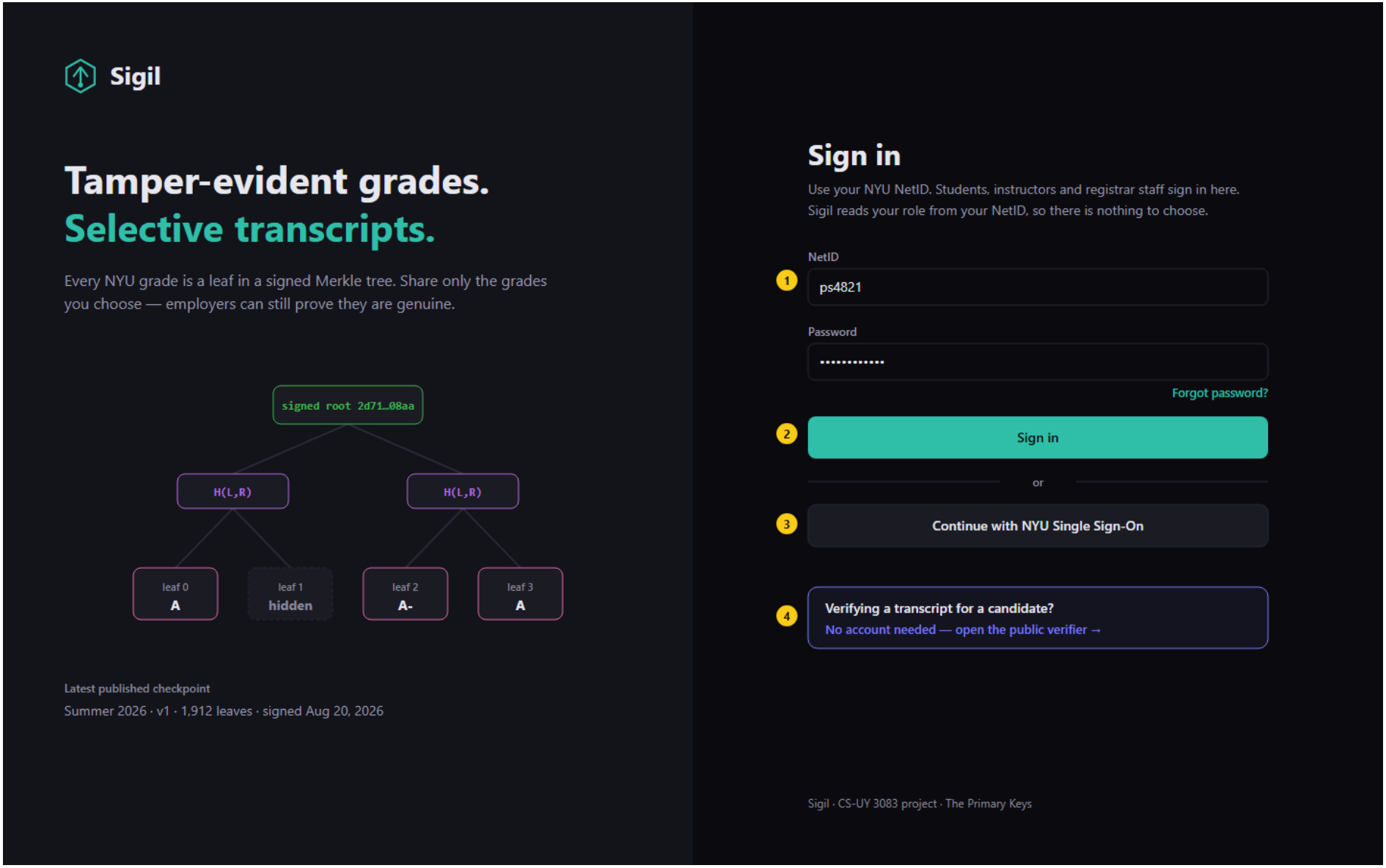
No.	Screen	User	Written by
10	Registrar: Correction approvals	Registrar staff	Mohammad Alyousef
11	Registrar: Publish Merkle checkpoint	Registrar staff	Mohammad Alyousef
12	Registrar: Audit ledger	Registrar staff	Mohammad Alyousef
13	Verifier: Verify a proof	Verifier (public, no login)	Harsh Agarwal
14	Verifier: Verification result	Verifier (public, no login)	Harsh Agarwal
15	Verifier: Checkpoint registry	Public	Harsh Agarwal
16	Mobile views: verifier results and sharing a proof	Verifier and Student (phone)	Harsh Agarwal

4. Screens and UI Components

Each screen has numbered yellow markers. The numbers match the rows in the table under the screen. Screens are printed on landscape pages at full width so their text stays readable.

Screen 1: Sign in

User: All signed-in users. Written by Pratyush Patel.



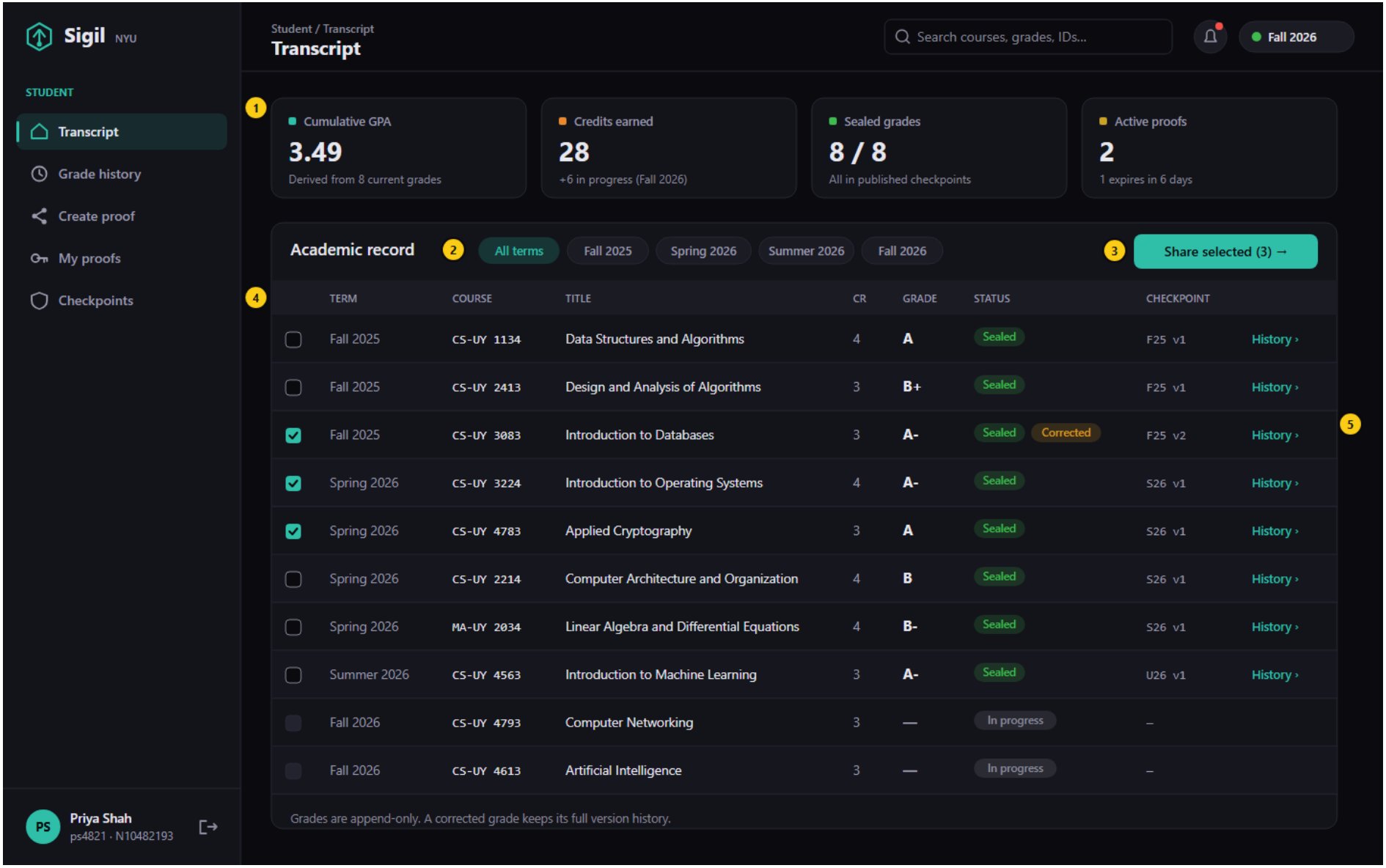
The single entry point for students, instructors and registrar staff, plus an obvious way out for verifiers, who never need an account.

Reads: University (name, latest MerkleCheckpoint for the footer). Auth is outside the schema.

No.	Component	Purpose	What it does	How users use it	What to expect
1	NetID and password fields	Authenticate with the user's NYU identity.	Takes the NetID and a masked password. 'Forgot password?' links to NYU's reset flow. There is no role picker: the role comes from the NetID, which removes a choice users could get wrong.	Type the NetID (e.g. ps4821) and password.	An inline error on a wrong password. Nothing is revealed about whether the NetID exists. After login they land on their role's home screen (Transcript, My courses or Dashboard).
2	Sign in button	Primary action.	Submits the credentials over HTTPS and starts a session.	Click it or press Enter.	Redirect to the role's home screen within a second.
3	NYU Single Sign-On	Alternative login that most NYU users already have.	Hands off to NYU SSO and returns with an authenticated session.	Click it instead of typing a password.	The same result as Sign in, with no Sigil password.
4	Verifier shortcut	Keeps employers off the login form.	Links to the public verifier (screen 13).	An employer clicks the link.	Verification starts right away. No account and no sign-up.

Screen 2: Student: Transcript

User: Student. Written by Khalid Alhabib.



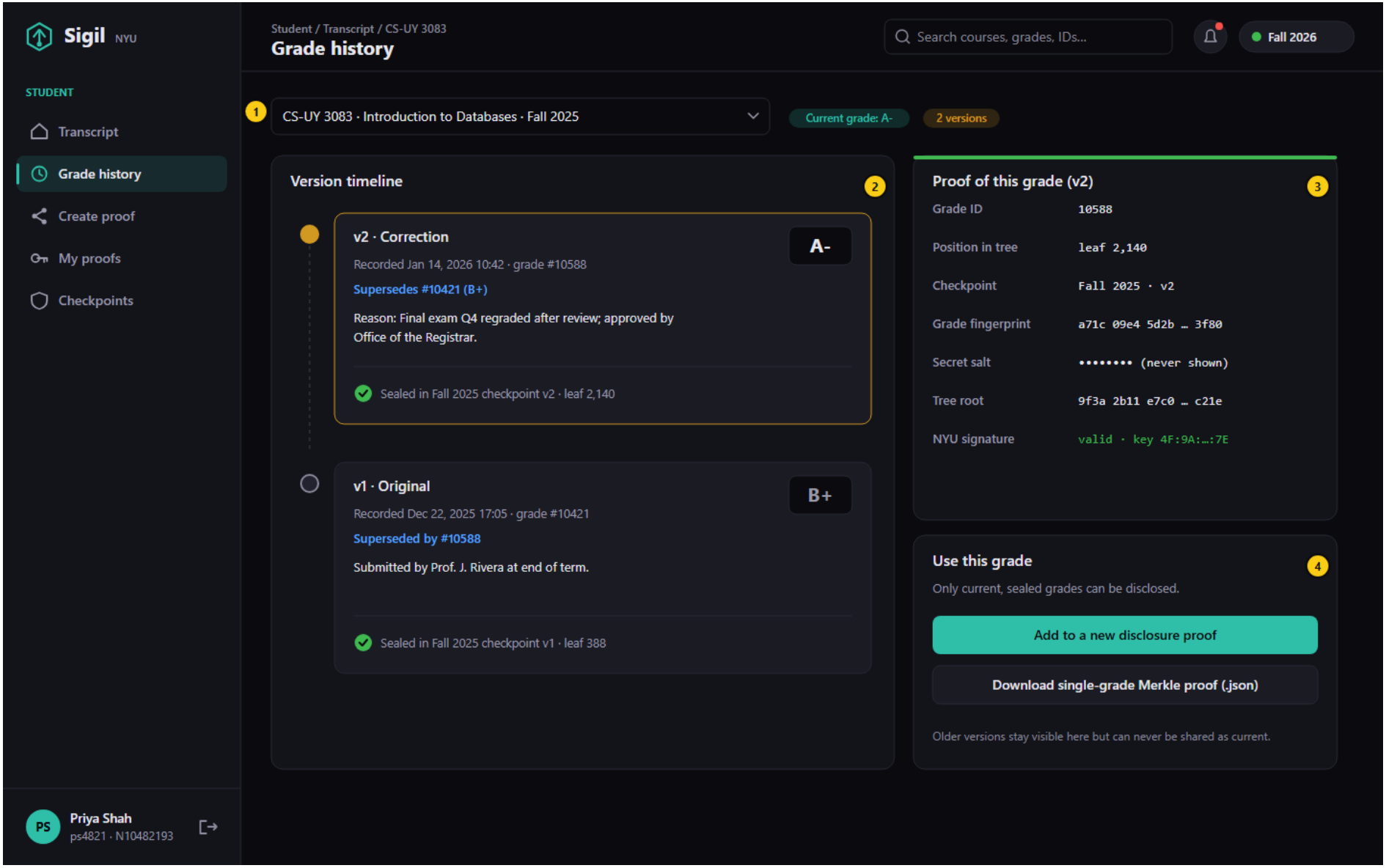
The student's home screen: the full academic record, each grade's integrity status, and the starting point for sharing grades.

Reads: Grade (current versions only), Course, MerkleCheckpoint, DisclosureProof (count). GPA is derived, not stored.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Summary tiles	A quick overview of the record.	Shows the derived GPA, credits earned and in progress, how many grades are sealed, and the number of active proofs.	Read at a glance. Each tile links to the related screen.	Numbers that always match the table below. GPA uses only current (non-superseded) grades.
2	Term filter chips	Narrow a long record.	Filters the table to one term or all terms.	Click a chip.	The table updates instantly. The selected chip is highlighted.
3	Share selected	Shortcut into proof creation.	Opens Create proof (screen 4) with the checked grades pre-selected.	Tick grades in the table, then click.	The proof wizard opens with those grades already chosen.
4	Academic record table	The authoritative list of the student's grades.	One row per course: term, code, title, credits, current letter grade, a status badge (Sealed or In progress), a separate Corrected tag when the grade has a newer version, and the checkpoint version it is sealed in. Unsealed rows have their checkbox disabled.	Scan, sort, and tick rows to share.	A corrected grade still counts as sealed (in its newer checkpoint version). Unsealed grades cannot be shared until the registrar publishes.
5	History link	Explains any grade.	Opens Grade history (screen 3) for that course.	Click 'History' on a row.	The complete version chain for that course and its proof data.

Screen 3: Student: Grade history

User: Student. Written by Khalid Alhabib.



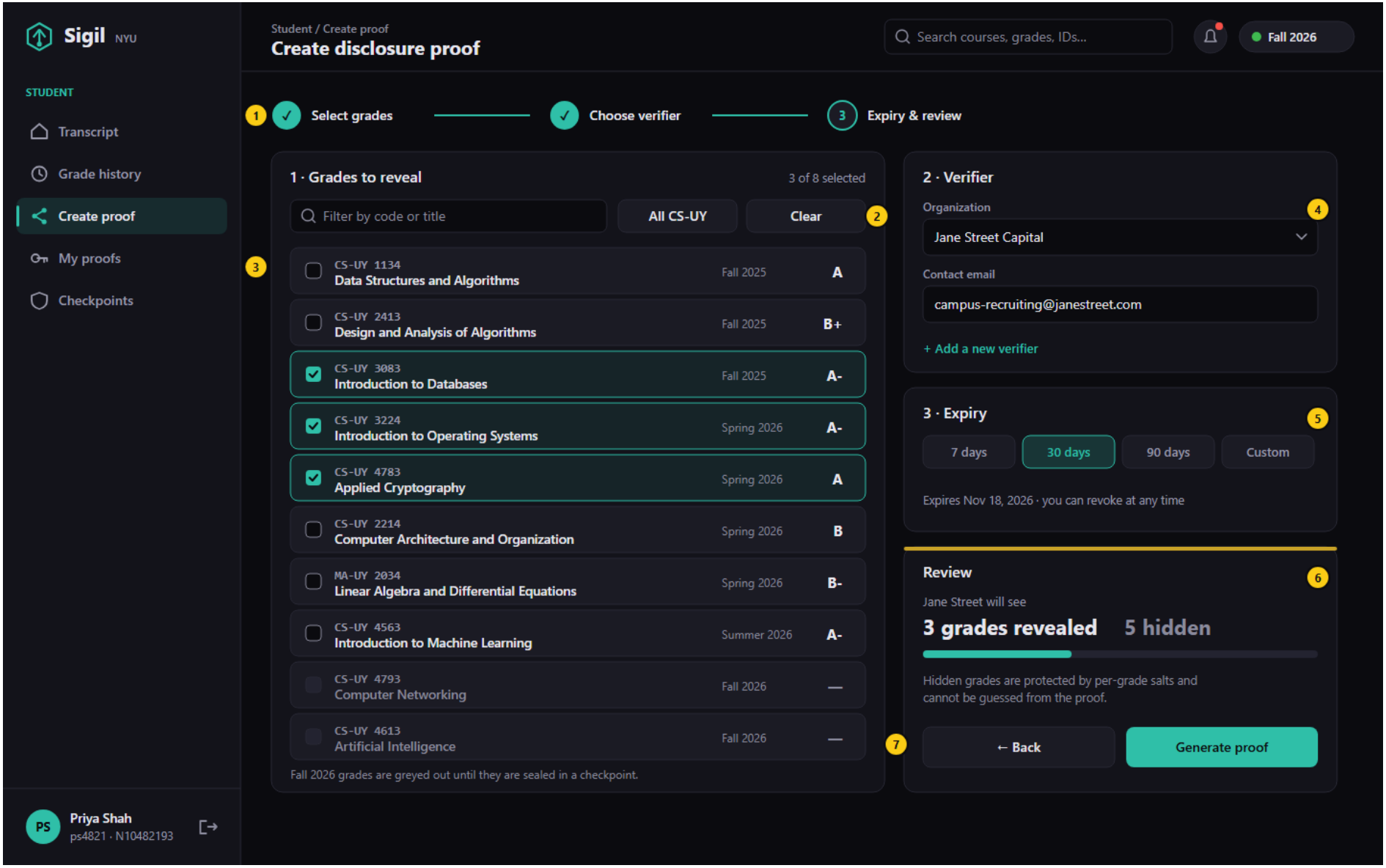
Makes the append-only rule visible: every version of a grade, why it changed, and the cryptographic evidence that seals it.

Reads: Grade (chain via supersedes_grade_id), MerkleCheckpoint, Instructor. The salt is never shown.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Course selector and badges	Choose which course's history to view.	A dropdown of the student's courses. Badges show the current grade and the version count.	Pick a course.	The timeline and leaf panel reload for that course.
2	Version timeline	Shows how the grade evolved.	Lists versions newest first. Each card shows the letter, recorded time, grade_id, which grade it supersedes or is superseded by, the change_reason, and the checkpoint/leaf it is sealed in.	Read top to bottom. Click a grade_id to jump to that version.	Nothing is ever hidden or overwritten. The original B+ is still listed under the corrected A-.
3	Proof of this grade	Proves the grade is part of a signed tree.	Shows the grade ID, its position in the tree (leaf_index), checkpoint, grade fingerprint (leaf_hash), tree root (root_hash) and signature status, in plain-language labels. The salt is masked.	Copy values, or show them to a skeptical party.	A green 'valid' signature means the grade matches NYU's published root.
4	Use this grade	Turns the history view into an action.	Adds the current grade to a new proof, or downloads its Merkle proof file.	Click one of the two buttons.	Superseded versions are not offered for sharing, only the current one.

Screen 4: Student: Create disclosure proof

User: Student. Written by Khalid Alhabib.



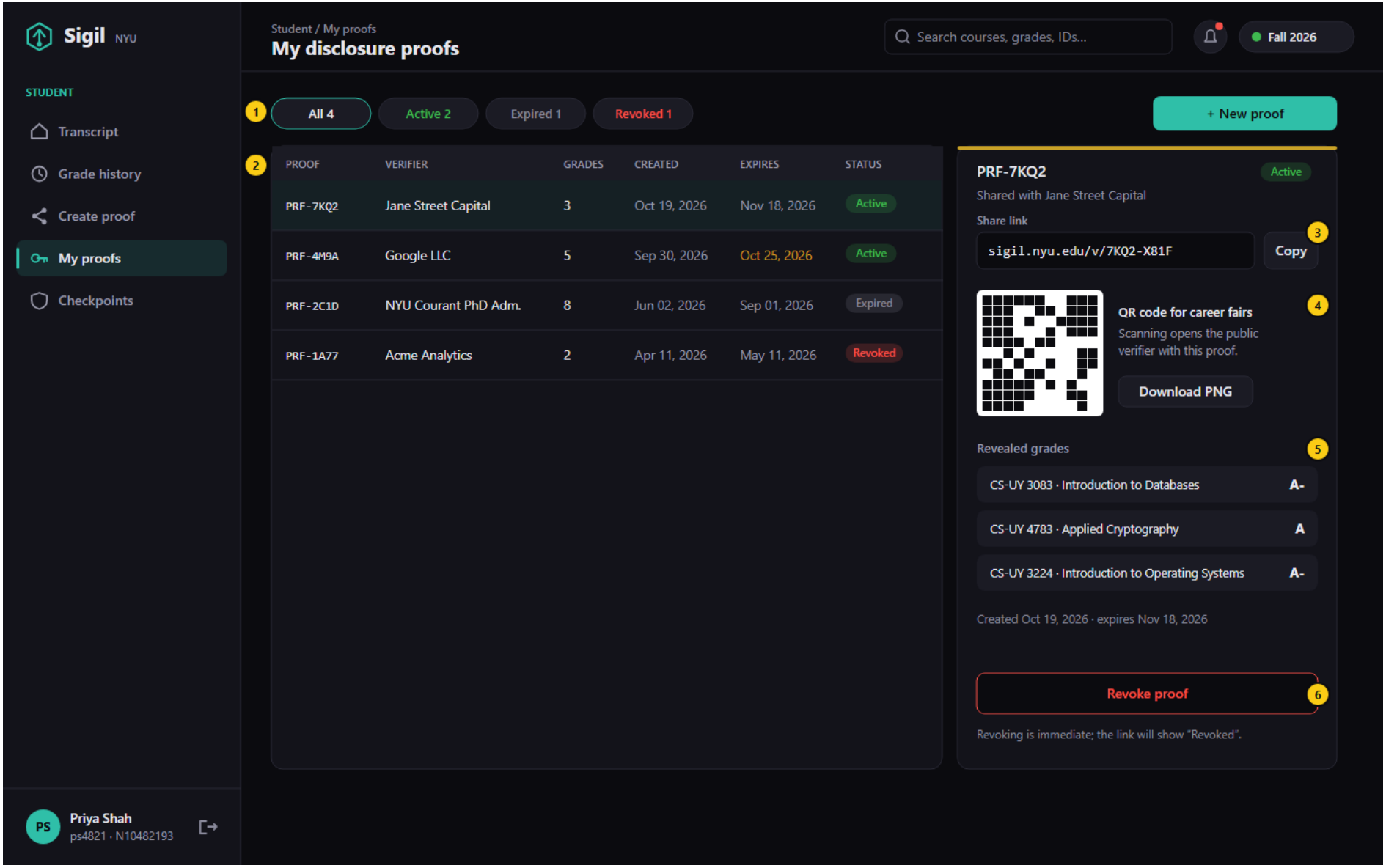
A guided, three-part form for building a selective-disclosure proof: which grades, for whom, and for how long.

Reads: Grade (current and sealed), Verifier. Writes: DisclosureProof, DisclosedGrade rows, and a Verifier row if new.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Progress stepper	Shows which parts of the one-page form are complete.	Three steps: Select grades, Choose verifier, Expiry & review. A step gets a check once its section is filled in. Here grades and verifier are done and the student is reviewing.	Read it, or click a step to scroll to that section.	Always clear what is left before generating.
2	Filter and quick-select	Speed up selection on long records.	Search by code or title. 'All CS-UY' ticks every CS course. 'Clear' resets.	Type or click.	Typical use: show only CS grades to a tech employer in one click.
3	Grade picker	Choose exactly what is revealed.	A checkbox row per grade. Selected rows are highlighted. Unsealed (Fall 2026) grades are greyed out and cannot be ticked.	Tick or untick rows.	The counter ('3 of 8 selected') and the Review panel update live.
4	Verifier	Records who receives the proof.	Pick an existing organisation from the dropdown or add a new one with a contact email.	Select or type.	The proof is bound to this verifier and its name shows on the proof page.
5	Expiry presets	Limit how long the proof works.	7, 30 or 90 days, or a custom date. Sets expires_at.	Click a preset.	The exact expiry date appears below. The proof can still be revoked earlier.
6	Review summary	Confirms the privacy outcome before committing.	Shows how many grades will be revealed versus hidden, as a bar and counts, with a note that salts protect the hidden ones.	Read before generating.	No surprises: what is shown here is exactly what the verifier sees.
7	Back / Generate proof	Navigation and the final action.	Generate builds the Merkle paths, creates the proof and opens it in My proofs.	Click Generate proof.	A share link and QR code appear within seconds.

Screen 5: Student: My proofs

User: Student. Written by Khalid Alhabib.



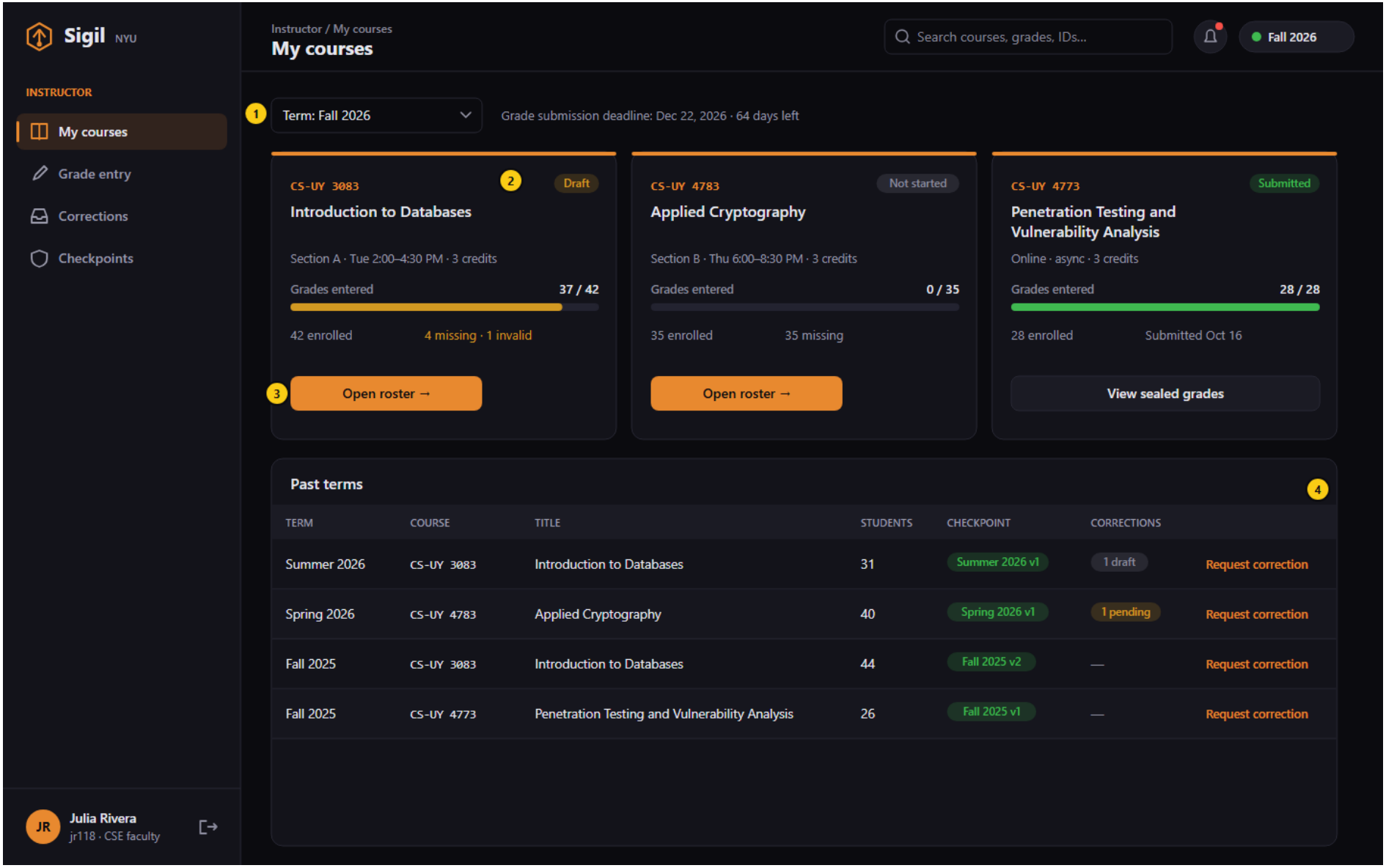
Lets students see, share and revoke every disclosure proof they have created.

Reads: DisclosureProof, Verifier, DisclosedGrade, Grade. Writes: DisclosureProof.status (active to revoked).

No.	Component	Purpose	What it does	How users use it	What to expect
1	Status tabs and New proof	Find proofs quickly.	Filters by All / Active / Expired / Revoked with counts. '+ New proof' opens screen 4.	Click a tab.	The list filters instantly.
2	Proofs table	Overview of everything shared.	Proof ID, verifier, number of grades, created and expiry dates (amber when expiring soon), and status badge.	Click a row to open it in the side panel.	The selected proof is highlighted and its details load on the right.
3	Share link and Copy	The main way to send a proof.	Shows the public URL (sigil.nyu.edu/v/…) and copies it to the clipboard.	Click Copy and paste it into an application or email.	The recipient opens the verifier with the proof preloaded.
4	QR code	Share in person (career fairs, interviews).	Encodes the same link. It can be downloaded as a PNG.	Show the QR code on a phone, or print it on a résumé.	Scanning opens the verification result page.
5	Revealed grades	A reminder of exactly what was disclosed.	Lists the course and grade for each revealed grade.	Read only.	Matches what the verifier sees. Hidden grades are not listed.
6	Revoke proof	Withdraw access at any time.	After a confirmation, sets the proof status to revoked.	Click and confirm.	The link immediately shows 'Revoked' to anyone who opens it. This cannot be undone.

Screen 6: Instructor: My courses

User: Instructor. Written by Pratyush Patel.



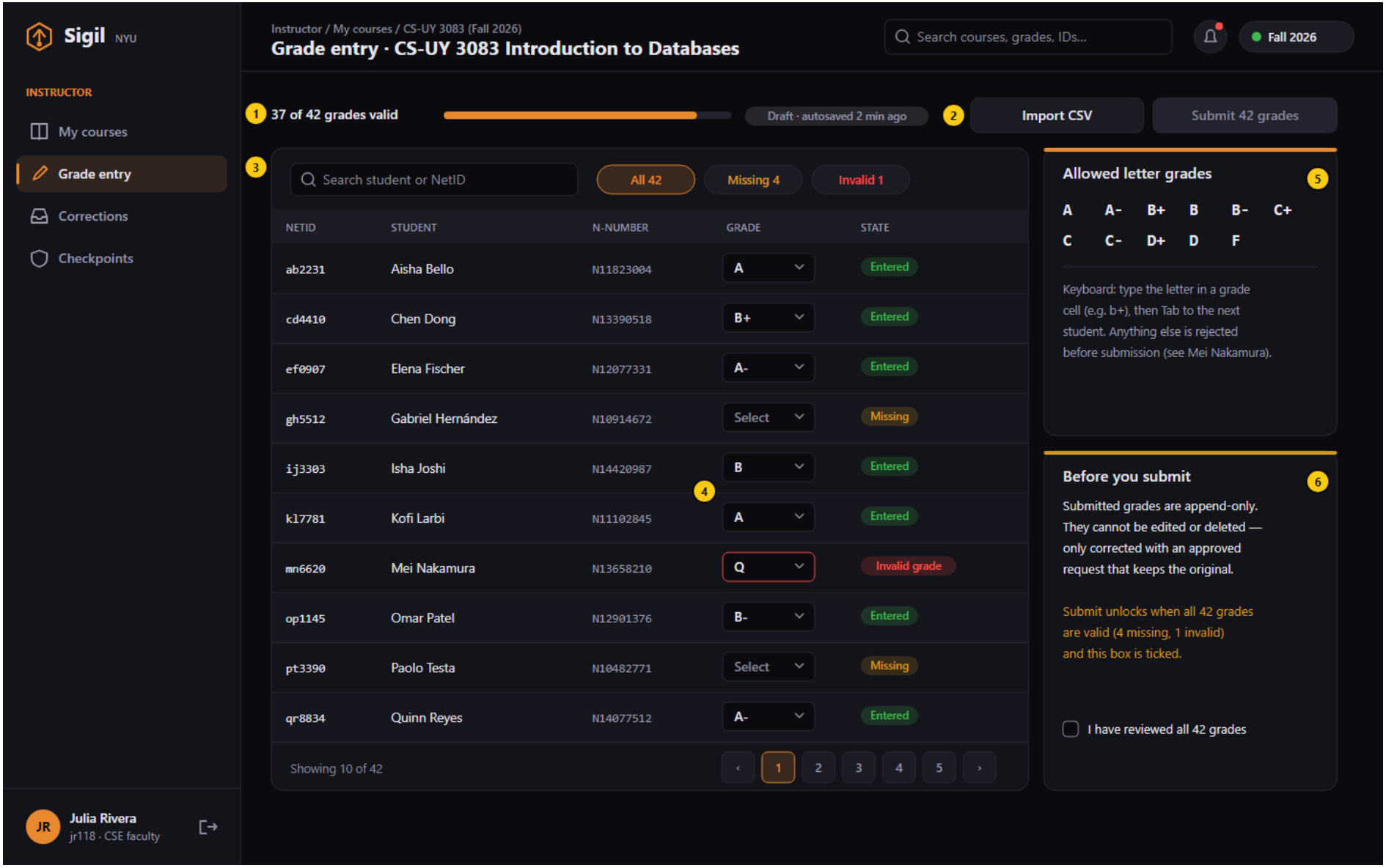
The instructor's home screen: courses taught this term, how far grading has got, and past terms for corrections.

Reads: Course (by instructor_id and term), Grade counts, MerkleCheckpoint.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Term selector and deadline	Focus on one term.	Switches terms and shows the grade submission deadline with days remaining.	Pick a term.	Course cards reload. The deadline stays visible as a reminder.
2	Course card	A status summary per course.	Shows code, title (wrapped, never cut off), section, credits, a status badge (Not started / Draft / Submitted), a grades-entered progress bar, enrolment, and what is still missing or invalid.	Scan the cards.	Unfinished courses stand out by colour and badge.
3	Open roster	Go to grading.	Opens Grade entry (screen 7) for that course. Submitted courses show 'View sealed grades' instead.	Click the button.	The roster opens with saved drafts kept.
4	Past terms table	Access to sealed courses.	Lists past courses with the checkpoint that sealed them, any draft or pending corrections, and a 'Request correction' link. Corrections only exist for sealed courses, so they appear here and not on the current-term cards.	Click Request correction on a row.	Opens screen 8 with the course pre-filled. The sealed grade itself can never be edited.

Screen 7: Instructor: Grade entry

User: Instructor. Written by Pratyush Patel.



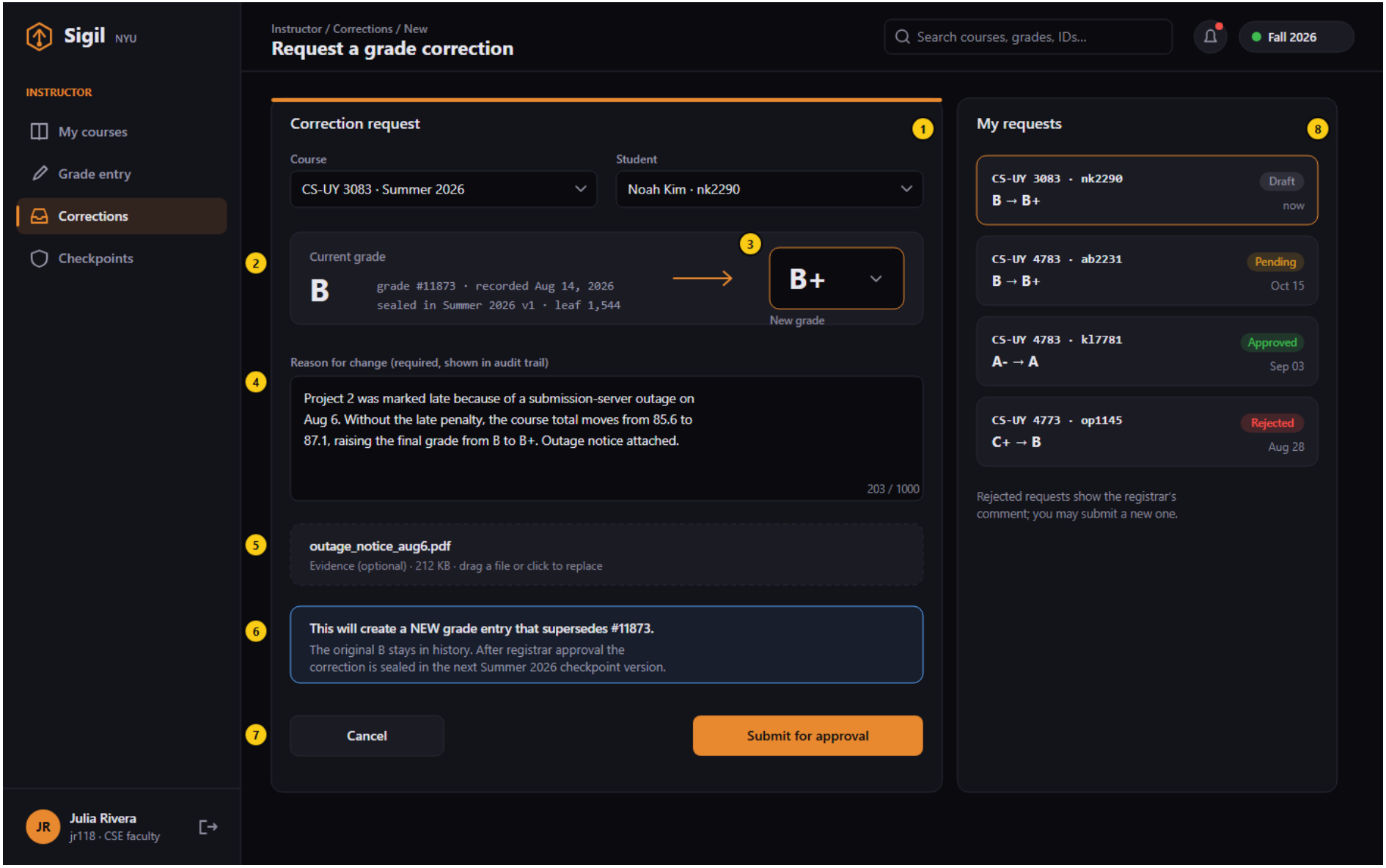
A fast, validated roster for entering final grades. Submitting turns them into permanent, append-only Grade rows.

Reads: Student (enrolled in course). Writes: Grade (original entries, one per student and course).

No.	Component	Purpose	What it does	How users use it	What to expect
1	Progress bar	Shows how much is left.	Counts valid grades out of enrolled students.	Read at a glance.	Reaches 100% only when every student has a valid grade.
2	Draft status, Import CSV, Submit	Save, bulk-load and commit.	Drafts autosave. Import CSV fills the roster from a spreadsheet. Submit writes all grades. In the mockup Submit is greyed out because 4 grades are missing, 1 is invalid and the box is unticked.	Import if helpful, then click Submit once everything is reviewed.	Submit stays disabled while any grade is missing or invalid, or the confirmation is unticked.
3	Search, filters and roster table	Find and grade each student.	Rows show NetID, name, N-number, a grade cell and a state badge (Entered / Missing / Invalid). 'Missing' and 'Invalid' filters jump straight to the rows that block submission. Paged, 10 per page, with 32 px page buttons.	Type a letter in the grade cell (or pick from the dropdown) and press Tab for the next student, so a whole roster can be graded from the keyboard.	Badges update as soon as a grade is entered.
4	Grade dropdown validation	Enforce the letter-grade business rule in the UI.	Anything outside A…F (e.g. 'Q' from a bad CSV) gets a red border and an 'Invalid grade' badge.	Fix the highlighted rows.	Invalid data never reaches the database.
5	Allowed letter grades	Reference card for the rule.	Lists the 11 permitted grades (A, A-, B+, B, B-, C+, C, C-, D+, D, F) as plain text, not buttons, because grades are entered in the roster. Also explains the keyboard shortcut.	Glance while grading.	The same list the database CHECK constraint enforces.
6	Before you submit	Explains permanence before it happens.	Explains that submitted grades are append-only and only correctable through approval. A required checkbox unlocks Submit.	Read it and tick the box.	No accidental submissions. Users know corrections need a reason and the registrar's approval.

Screen 8: Instructor: Request a grade correction

User: Instructor. Written by Pratyush Patel.



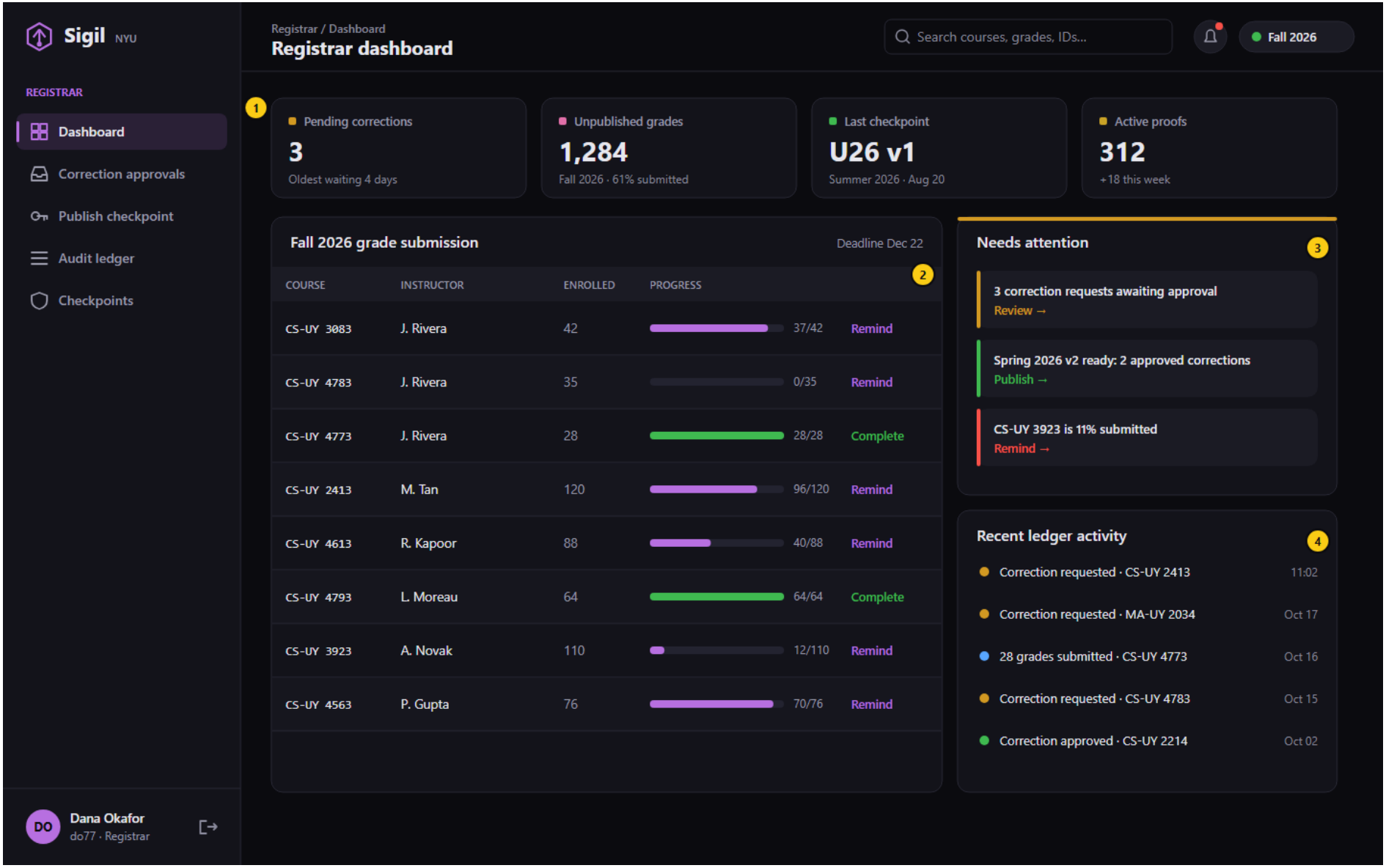
The only way to change a recorded grade: a new superseding entry with a mandatory reason, sent to the registrar for approval.

Reads: Grade (current version). Writes: a pending correction which, once approved, becomes a new Grade with supersedes_grade_id and change_reason.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Course and student selectors	Identify the grade being corrected.	Dropdowns limited to the instructor's own courses and enrolled students.	Pick the course, then the student.	Only grades they are allowed to correct can be chosen (same university, own course).
2	Current grade panel	Show exactly what will be superseded.	Displays the current letter, grade_id, recorded date, checkpoint and leaf.	Read to confirm it is the right record.	The grade_id here becomes supersedes_grade_id.
3	New grade	Choose the corrected value.	A dropdown with only valid letter grades.	Select the new grade.	The arrow shows the change (B to B+).
4	Reason for change	Satisfies the business rule that every correction is justified.	A required text area with a character counter. The text is stored as change_reason.	Describe why the grade changed.	The reason is shown to the registrar, the audit ledger and the student's history.
5	Evidence upload	Supporting documentation.	Optional file attachment (e.g. the re-graded exam).	Drag a file in or click to browse.	The registrar sees the file during approval.
6	Consequence notice	Sets expectations.	Explains that a new entry will be created, the original stays in history, and approval adds it to the next checkpoint version.	Read.	No belief that the old grade is 'edited'.
7	Cancel / Submit for approval	Final actions.	Submit sends the request to the registrar queue.	Click Submit.	The request appears under My requests as Pending.
8	My requests	Track outcomes.	Lists the instructor's requests with status (Draft, Pending, Approved, Rejected) and date.	Click a request to view it or the registrar's comment.	Rejected requests show the reason and can be resubmitted as a new request.

Screen 9: Registrar: Dashboard

User: Registrar staff. Written by Mohammad Alyousef.



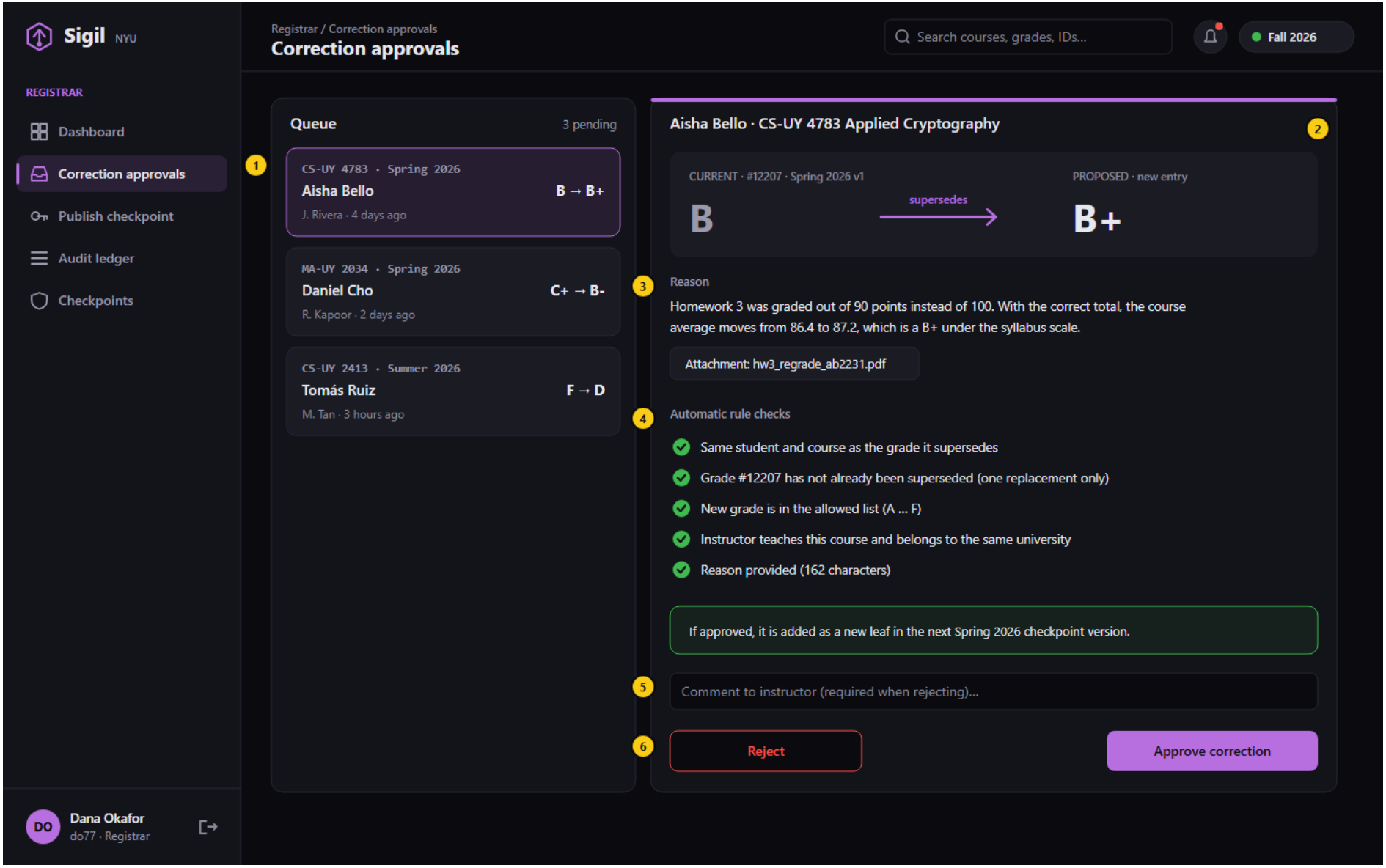
Gives the registrar one view of this term's grading, what needs attention, and recent ledger events.

Reads: Grade, Course, Instructor, MerkleCheckpoint, DisclosureProof (aggregates).

No.	Component	Purpose	What it does	How users use it	What to expect
1	KPI tiles	Status at a glance.	Shows pending corrections, unpublished grades this term, the last checkpoint, and active proofs.	Read. Click a tile to open the matching screen.	Exact counts. The oldest pending correction age is shown.
2	Grade submission table	Track instructors before the deadline.	Per-course enrolment and a progress bar of grades entered, with a 'Remind' action.	Scan for low progress and click Remind.	Complete courses turn green. Laggards stand out.
3	Needs attention	A prioritised to-do list.	Shows items such as corrections waiting, a checkpoint version ready to publish, and very late courses.	Click the item's link.	Takes them straight to the relevant screen.
4	Recent ledger activity	Situational awareness.	A time-ordered feed of approvals, requests, submissions, revocations and publications.	Read. Click an entry to open it in the audit ledger.	A chronological view of everything written to the ledger.

Screen 10: Registrar: Correction approvals

User: Registrar staff. Written by Mohammad Alyousef.



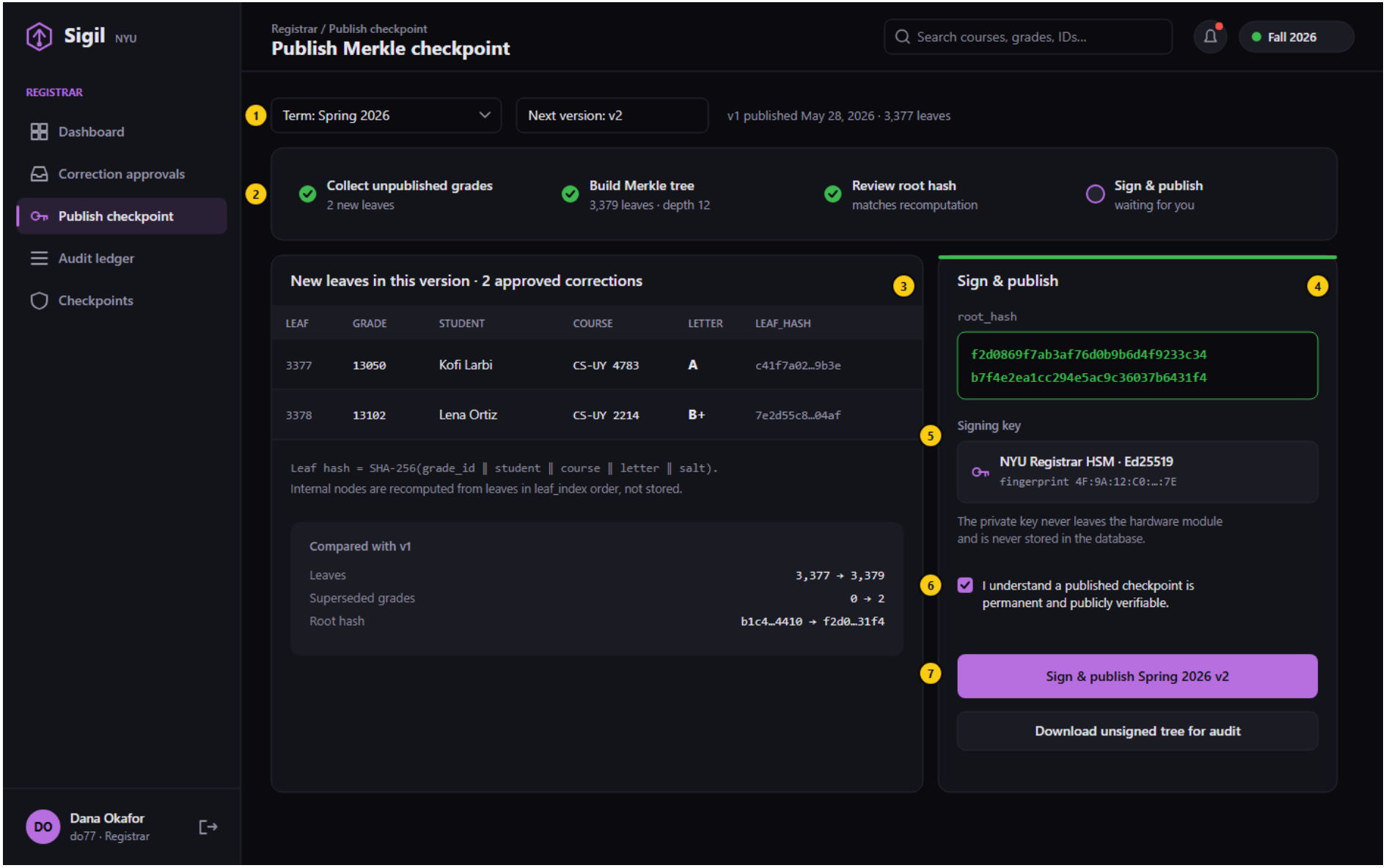
The authorised-personnel checkpoint from Milestone 1: no grade changes without a registrar reviewing and approving a superseding entry.

Reads: pending corrections, Grade, Instructor, Course. Writes: approved Grade row (append) or a rejection with comment.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Request queue	Work through requests in order.	Lists pending requests with course, term, student, the proposed change and who asked, and when.	Click a request.	It opens on the right. The queue shrinks as items are decided.
2	Change summary	Shows exactly what will happen.	Shows CURRENT (#12207, B, sealed in Spring 2026 v1) superseded by PROPOSED (B+) as a new entry.	Read.	Makes clear that a new row is added and nothing is overwritten.
3	Reason and attachment	Evidence for the decision.	Shows the instructor's change_reason and any uploaded file.	Read and open the file.	Enough context to decide without email back-and-forth.
4	Automatic rule checks	Enforce the business rules before a human decides.	Checks same student and course, the original not already superseded (one replacement only), valid letter, instructor teaches the course in the same university, reason present.	Review the green ticks. A failed rule shows a red cross and blocks approval.	Invalid corrections cannot be approved by mistake.
5	Comment field	Talk to the instructor.	Free text sent back with the decision. Required when rejecting.	Type a note.	The instructor sees it under My requests.
6	Reject / Approve correction	Make the decision.	Approve appends the new Grade row and queues it for the next checkpoint version. Reject closes the request.	Click one.	Immediate confirmation. Approved items show up in the publish screen and the audit ledger.

Screen 11: Registrar: Publish Merkle checkpoint

User: Registrar staff. Written by Mohammad Alyousef.



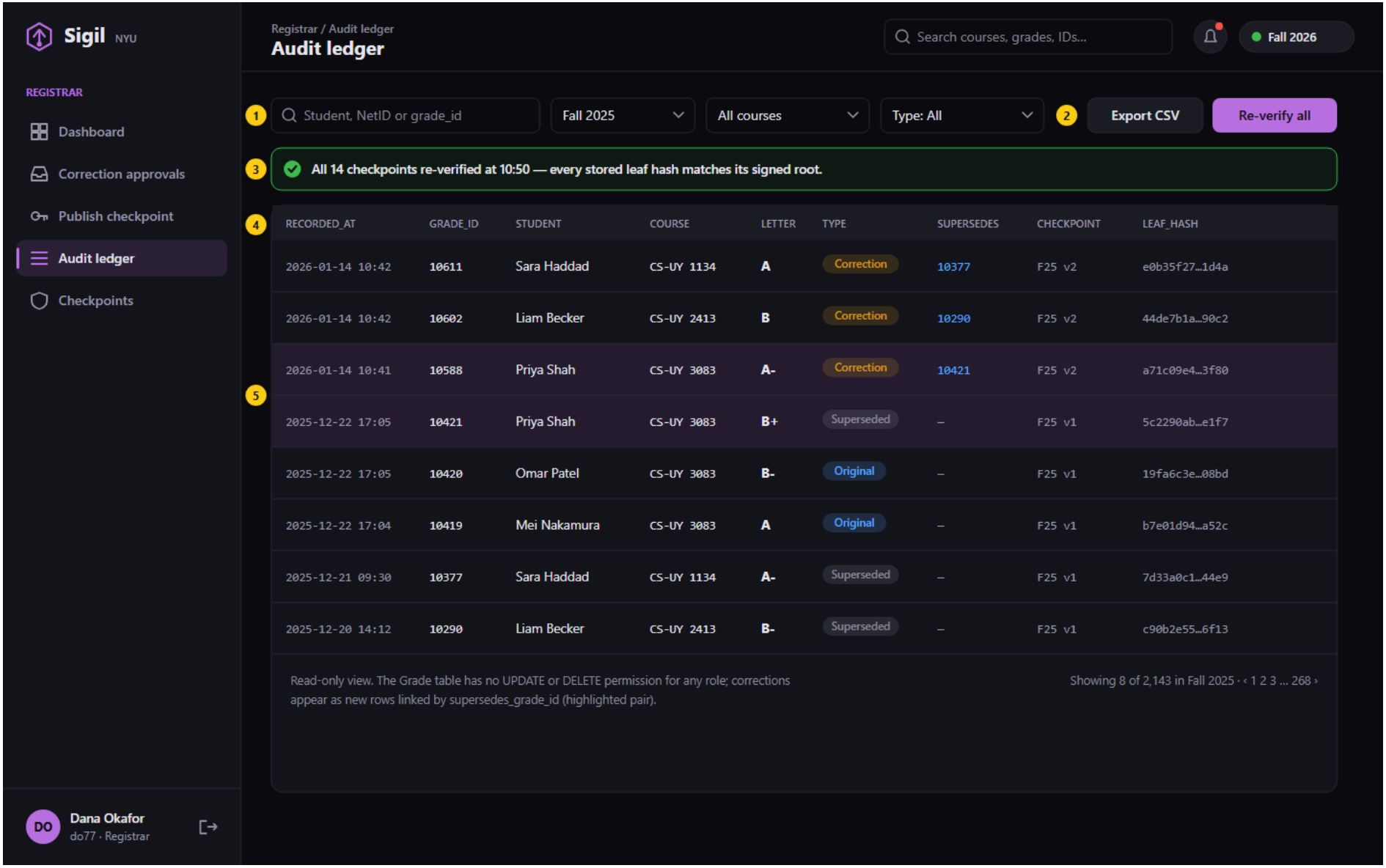
Seals all unpublished grades for a term into a Merkle tree and publishes the signed root that everyone verifies against.

Reads: Grade (checkpoint_id IS NULL for the term). Writes: MerkleCheckpoint (root_hash, leaf_count, signature, version_no) and sets checkpoint_id and leaf_index on those Grades.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Term and version	Choose what to publish.	Pick the term. The next version number is filled in, with the previous version's date and leaf count shown.	Select a term.	Corrections to an old term produce v2, v3 and so on, never an edit.
2	Pipeline steps	Make the cryptographic process understandable.	Collect unpublished grades, Build Merkle tree, Review root hash, Sign & publish, each with a check or a waiting state.	Read.	Clear progress. Signing is only possible after the review.
3	New leaves table and comparison	Review exactly what is being added.	Lists each new leaf (index, grade_id, student, course, letter, leaf_hash), the hash formula, and a comparison with the previous version.	Scan for anything unexpected.	Exactly the approved corrections appear. Nothing else changes.
4	Root hash	The value being signed.	Shows the full 256-bit root (64 hex characters) in monospace.	Compare with an independent recomputation if desired.	The same root later appears in the public registry.
5	Signing key	Shows which key signs, and that it stays protected.	Shows the HSM key name, algorithm and fingerprint, and states that the private key is never stored in the database.	Read.	Confidence that the business rule on key storage is met.
6	Permanence confirmation	Prevent accidental publication.	A required checkbox acknowledging that a checkpoint is permanent and public.	Tick it.	Unlocks the publish button.
7	Sign & publish / Download tree	Commit, or export for audit.	Signs the root, writes the MerkleCheckpoint, and links the leaves. The secondary button exports the unsigned tree.	Click Sign & publish.	The new version is live in the registry within seconds and student statuses change to Sealed.

Screen 12: Registrar: Audit ledger

User: Registrar staff. Written by Mohammad Alyousef.



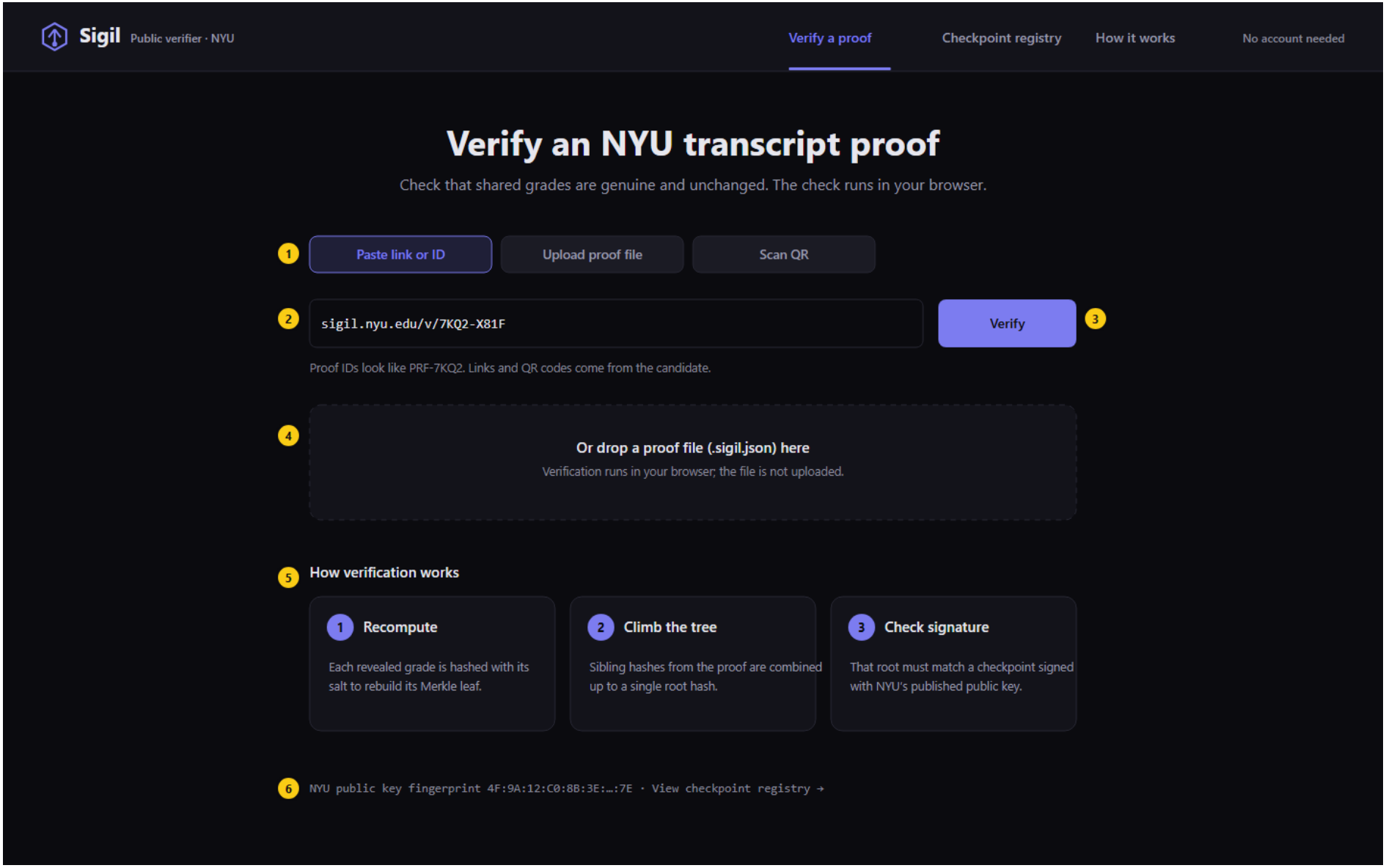
A read-only, searchable view of every Grade row ever written, with a one-click integrity check of all checkpoints.

Reads: Grade (all versions), Student, Course, MerkleCheckpoint. No writes; the table has no UPDATE/DELETE grant.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Search and filters	Find any record.	Free-text search by student, NetID or grade_id, plus term, course and type (Original / Correction / Superseded) filters.	Type or pick filters.	Results narrow instantly.
2	Export CSV / Re-verify all	Reporting and integrity.	Export downloads the filtered ledger. Re-verify recomputes every leaf hash and root against the signed checkpoints.	Click either button.	A CSV file, or a pass/fail integrity result in the banner.
3	Integrity banner	Proof that nothing was tampered with.	Shows the result and time of the last re-verification.	Read.	Green when every stored leaf matches its signed root. Any mismatch is shown in red with the affected rows.
4	Ledger table	The complete history.	One row per Grade: recorded_at, grade_id, student, course, letter, type badge, supersedes, checkpoint, leaf_hash.	Scroll, page, and click a grade_id to follow its chain.	Old values are never removed. Superseded rows stay visible.
5	Linked correction pair	Make supersession easy to see.	Highlights a correction and the original it supersedes together (e.g. #10588 and #10421).	Hover or click either row.	The full before/after picture in one place.

Screen 13: Verifier: Verify a proof

User: Verifier (public, no login). Written by Harsh Agarwal.



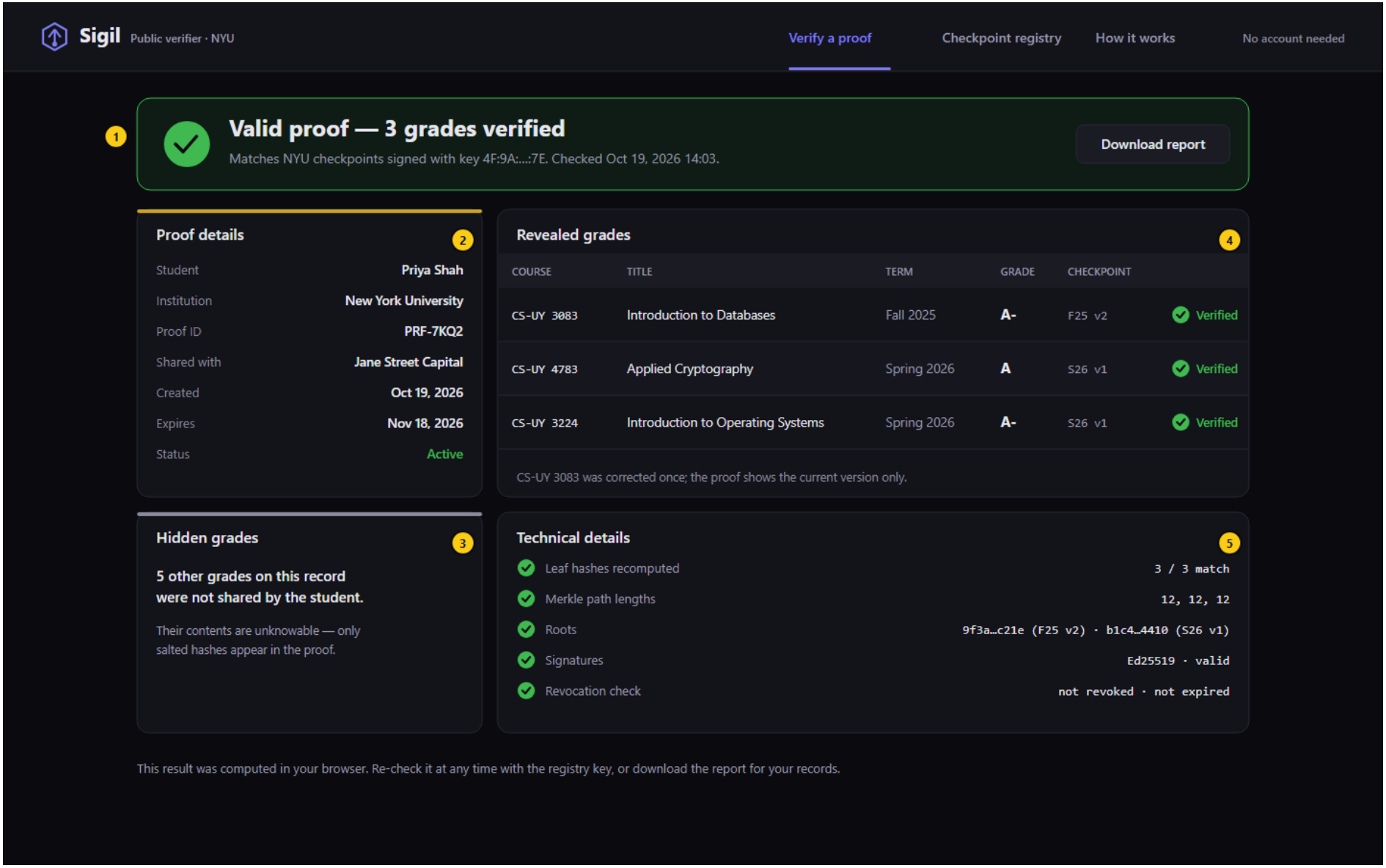
The public entry point for employers and admissions offices to check a proof with no account.

Reads: DisclosureProof (by ID), DisclosedGrade, MerkleCheckpoint, University.public_key.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Input method tabs	Accept a proof however it arrived.	Switches between pasting a link or ID, uploading a proof file, and scanning a QR code.	Click the tab that matches what they received.	The input area changes to suit the method.
2	Link / ID field	Main input.	Accepts a sigil.nyu.edu/v/… link or a PRF-xxxx ID.	Paste it in.	The format is checked as they type.
3	Verify button	Start the check.	Fetches the proof and runs verification in the browser.	Click Verify.	The result page (screen 14) loads in about a second.
4	Proof file drop zone	Offline or emailed proofs.	Accepts a .sigil.json file. Verification runs locally and the file is not uploaded.	Drag the file in.	The same result page, with no data sent to NYU.
5	How verification works	Build trust in the result.	Three plain-language steps: recompute leaves, climb the tree, check the signature.	Read.	Non-technical verifiers understand why a 'Valid' result can be trusted.
6	Public key fingerprint	Root of trust.	Shows NYU's signing-key fingerprint and links to the checkpoint registry.	Compare it with NYU's published value if desired.	Independent assurance.

Screen 14: Verifier: Verification result

User: Verifier (public, no login). Written by Harsh Agarwal.



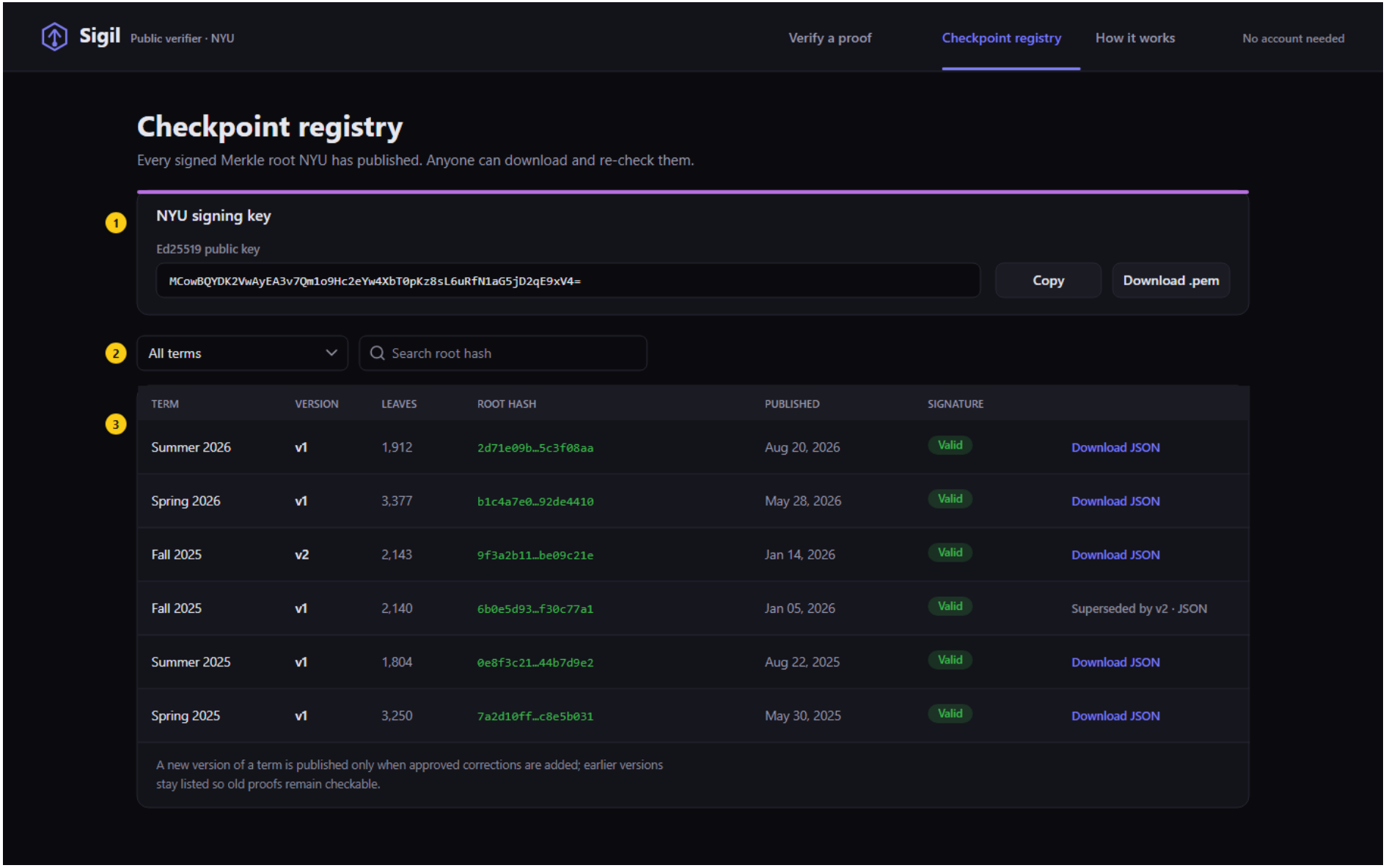
A clear answer on whether the revealed grades are genuine, current and still valid, with the evidence behind it.

Reads: DisclosureProof (status, expires_at), Verifier, DisclosedGrade, Grade, MerkleCheckpoint, University.

No.	Component	Purpose	What it does	How users use it	What to expect
1	Verdict banner	The answer first.	A large green 'Valid proof' (or red or amber for other states) with the number of grades verified, the signing key and the check time. 'Download report' saves a PDF.	Read. Download if a record is needed.	No interpretation needed.
2	Proof details	Context.	Student, institution, proof ID, who it was shared with, created and expiry dates, status.	Read.	Confirms the proof was meant for them and is in date.
3	Hidden grades notice	Honest about what is not shown.	States how many grades the student withheld and that they cannot be inferred.	Read.	Understands that this is a partial transcript by design and still authentic.
4	Revealed grades table	The verified data.	Course, title, term, grade, checkpoint, and a per-grade 'Verified' tick.	Read.	Each grade is individually proven. A corrected course shows only its current version.
5	Technical details	Evidence for technical reviewers.	Leaf hash matches, Merkle path lengths, roots, signature algorithm, revocation and expiry check.	Expand or read.	Full transparency about how the result was reached. Failure states (Revoked, Expired, Tampered, Unknown) have their own layout on screen 16, so a valid result never shows red.

Screen 15: Verifier: Checkpoint registry

User: Public. Written by Harsh Agarwal.



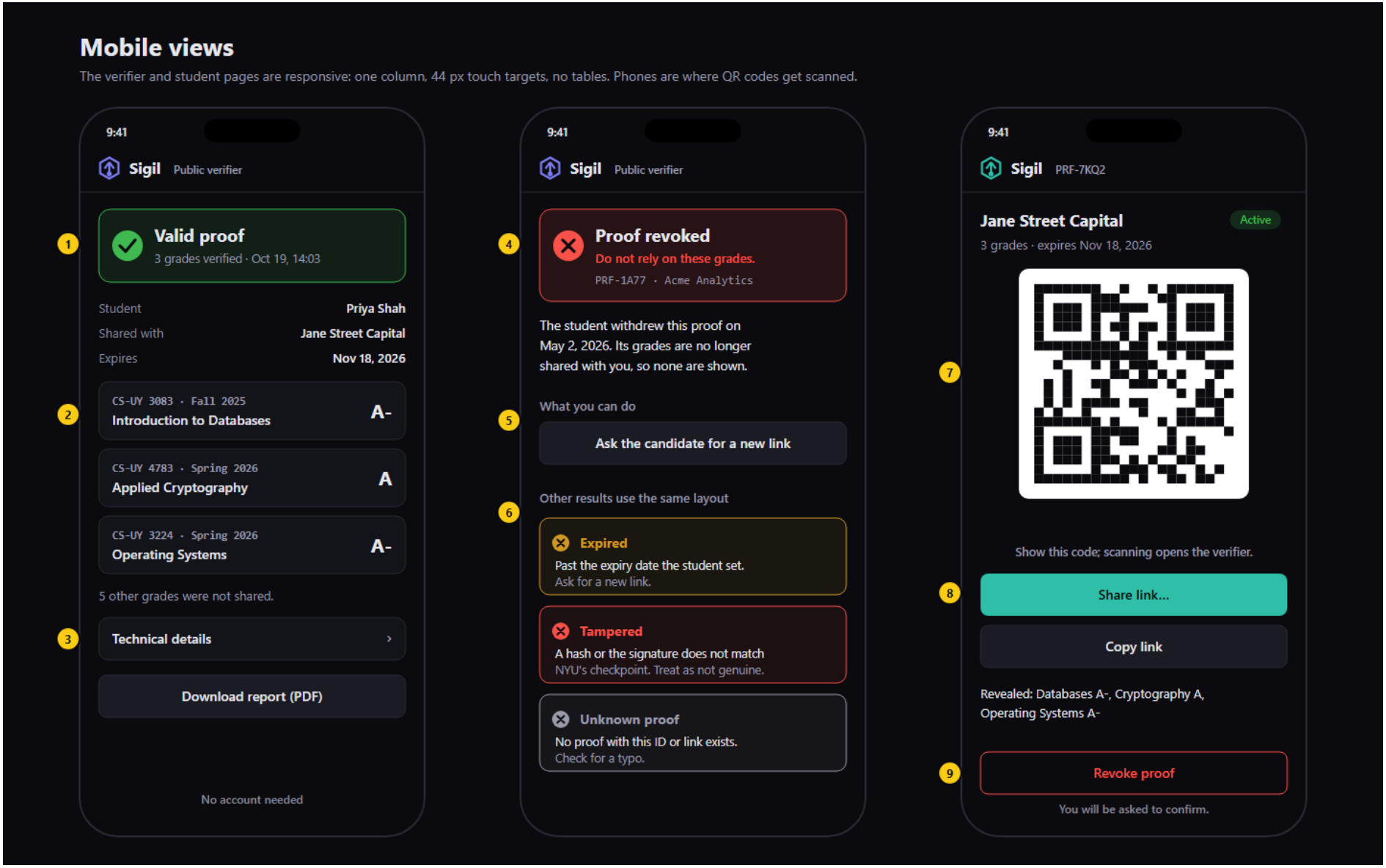
Publishes every signed Merkle root and the university public key, so anyone can audit NYU independently.

Reads: MerkleCheckpoint (all versions), University.public_key.

No.	Component	Purpose	What it does	How users use it	What to expect
1	NYU signing key	Root of trust for all verification.	Displays the Ed25519 public key with Copy and Download .pem.	Copy or download.	A key they can use with any standard crypto tool.
2	Term filter and root search	Find a specific checkpoint.	Filter by term or search by root hash prefix.	Type or select.	The table narrows instantly.
3	Checkpoint table	The public history of every seal.	Term, version, leaf count, root hash, publish date, signature validity, and a JSON download. Superseded versions are labelled but kept.	Download a checkpoint to re-verify it themselves.	Old versions never disappear, so older proofs remain checkable.

Screen 16: Mobile views: verifier results and sharing a proof

User: Verifier and Student (phone). Written by Harsh Agarwal.



QR codes are scanned on phones, so the verifier result and the student's share page are drawn at phone width. This screen also shows the failure states a verifier can get.

Reads: DisclosureProof (status, expires_at), DisclosedGrade, Grade, MerkleCheckpoint, University.public_key. Writes: DisclosureProof.status (revoke).

No.	Component	Purpose	What it does	How users use it	What to expect
1	Verdict banner (valid)	The answer first, on a small screen.	A green check with 'Valid proof', the grade count and the check time.	Read.	The same verdict as screen 14, readable without zooming.
2	Stacked grade cards	Tables do not fit on a phone.	Each revealed grade becomes a card: course, term, title and grade.	Scroll.	Nothing is cut off or scrolls sideways.
3	Collapsed technical details	Keeps the screen short for non-technical readers.	Hash, path and signature checks sit behind one 44 px row.	Tap to expand.	Recruiters see the answer, and engineers can still see the evidence.
4	Failure banner (revoked)	Make a failed check unmistakable.	A red cross, 'Proof revoked', 'Do not rely on these grades', and the proof ID. No grades are shown.	Read.	No chance of mistaking a revoked proof for a valid one.
5	Next step	Help the verifier recover.	A single action: ask the candidate for a new link.	Tap it to copy a ready-made message.	The verifier knows what to do, not just that something failed.
6	Other failure states	Cover every outcome the verifier can see.	Expired (amber), Tampered (red) and Unknown proof (grey) use the same layout, each with its own reason and next step.	Read.	Each failure explains its cause in plain language (heuristic: help users recover from errors).
7	Full-screen QR code	Share in person at a career fair or interview.	Shows the proof's QR code large enough to scan from across a table.	Hold the phone up to the recruiter.	Scanning opens the verification result.
8	Share link / Copy link	Send the proof from the phone.	'Share link…' opens the phone's native share sheet (Web Share API). 'Copy link' copies the URL.	Tap one.	The link can go to email, LinkedIn or a messaging app.
9	Revoke proof	Withdraw access on the go.	Red outlined button. A confirmation dialog follows, as on screen 5.	Tap and confirm.	The link shows 'Revoked' immediately, like the middle phone.